

LC 1547: Minimum cost to cut a stick

Authored by: Arpit Raj, LNMIIT Jaipur | **Date:** September 1, 2026

Copyright © 2026 Arpit Raj. All rights reserved.

Media.Net, PhonePe, Adobe

Problem Understanding

A stick of length n , labelled 0 to n .



0 to 4 label | length = 4

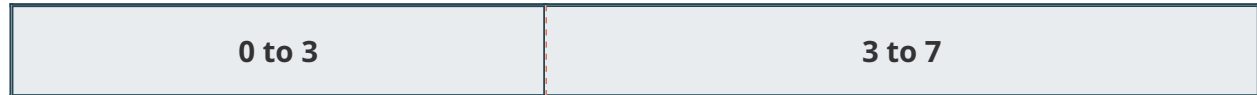
Now cuts is a vector, need to arrange it optimally such that sum of costs of cut is minimized, need to return min cost.

Cost of 1 cut is length of part where cut is made.

Visual Example: Step-by-Step Cuts

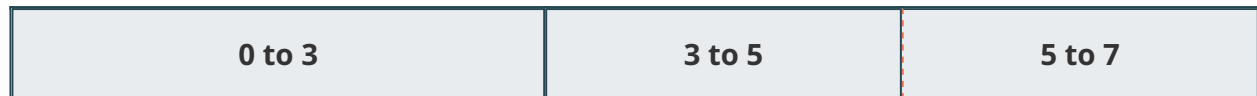
Let optimal cuts = [3, 5, 1, 4] on 0 to 7

1. Initial Stick (0 to 7) → Cut at 3:



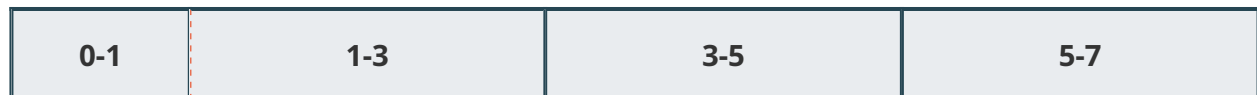
$$C1 = 7 - 0 = 7$$

2. Cut at 5 (falls in 3 to 7 piece):



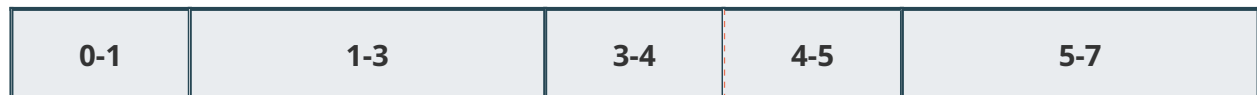
$$C2 = 7 - 3 = 4$$

3. Cut at 1 (falls in 0 to 3 piece):



$$C3 = 3 - 0 = 3$$

4. Cut at 4 (falls in 3 to 5 piece):



$$C4 = 5 - 3 = 2$$

Hence total cost = $7 + 4 + 3 + 2 = 16$

Interval DP Approach

Looks like interval DP. For points $0 \dots n$ and cuts c_1, c_2, c_3 :

Don't think which cut to perform first. Instead think which cut is last.

So for an interval $[l \dots j] [j \dots r]$:

The cost of making this cut j is $(r - l)$.

$$dp[l][r] = \min (dp[l][j] + dp[j][r] + (r-l))$$

Where (1) is Left Subproblem, (2) is Right Subproblem, and (3) is cost of current cut.

Hence go from smaller intervals to bigger intervals. Process smaller intervals first and then bigger intervals.

Meaning of DP state:

$dp[l][r]$ is basically cuts $[l] \dots \dots \dots$ cuts $[r]$ is interval ke andar jitne cuts hain, unko perform karne ka min cost.

So cuts will be sorted and 0 and n will also be included as they are boundaries.

Base Case

Agar l aur r ke beech koi cut nahi to **cost = 0**.
Hence adjacent boundaries ke beech koi cut nahi.

Eg: cuts = $[0, 1, 3, 4]$ then $dp[0][1] = 0$, $dp[1][2] = 0$.
Indices of cuts in cuts, so $dp[1][2]$ means no cut bw (1 and 3).

C++ Implementation

```
class Solution {
public:
    int minCost(int n, vector<int>& cuts) {
        cuts.push_back(0);
        cuts.push_back(n);

        sort(cuts.begin(), cuts.end());
        int m = cuts.size();

        // dp[l][r] == min cost of making all cuts between l and r
        vector<vector<int>> dp(m, vector<int>(m, INT_MAX));
        // m size as we are only interested in intervals with cuts
        // base case there is no cut between adjacent cuts

        for(int i = 0; i < m - 1; i++) {
            dp[i][i+1] = 0; // len==1 case , i.e 0 index cut with 1 index
                           // 1 index cut with 2 index
                           // 2 index cut with 3 index
                           // so on
        }

        for(int len = 2; len < m; len++) {
            for(int l = 0; l + len < m; l++) {

                int r = l + len;
                int left = cuts[l];
                int right = cuts[r];

                // [l.....j][j.....r]
                for(int j = l + 1; j < r; j++) {
                    if(dp[l][j] == INT_MAX) continue;
                    if(dp[j][r] == INT_MAX) continue;

                    dp[l][r] = min(dp[l][r], dp[l][j] + dp[j][r] + right -
left);
                }
            }
        }

        return dp[0][m-1];
    }
};
```